

目录

目录

要点速背

- 0.1 高频要点
- 0.2 高频公式 / 关键结论

用户模式运行ARM程序

- 0.3 易混淆点

第 1 章 嵌入式系统绪论

- 1.3 嵌入式系统结构
- 1.4 嵌入式操作系统

第 2 章 Linux 基本操作

- 2.1 Linux 目录结构
- 2.2 常用命令 (※)
- 2.3 路径与特殊符号 (※)
- 2.4 权限管理 (※)
- 2.5 管道、重定向与vi编辑器

第 3 章 Shell 编程

- 3.1 Shell 概述
- 3.2 变量 (※)
- 3.3 算术运算 (※)
- 3.4 条件判断 (※)
- 3.5 控制流
- 3.6 函数与参数传递

第 4 章 文件 IO 编程

- 4.1 文件流与标准流
- 4.2 POSIX 文件 IO (系统调用) (※)
- 4.3 C 语言库函数 IO (※)
- 4.4 时间编程 (※)

第 5 章 进程

- 5.1 基本概念 (※)
- 5.2 进程特性与状态
- 5.3 进程组成
- 5.4 进程管理命令
- 5.5 进程控制编程 (※)
- 5.6 任务分类

第 6 章 进程间通信

- 6.1 IPC 概述
- 6.2 管道 (Pipe) (※)
- 6.3 FIFO (命名管道) (※)
- 6.4 消息队列 (※)
- 6.5 信号 (Signal) (※)
- 6.6 信号量 (Semaphore) (※)
- 6.7 共享内存

第 7 章 Linux 网络编程

- 7.1 网络程序基础
- 7.2 网络地址与配置 (※)
- 7.3 网络协议 (※)
- 7.4 Socket 网络编程函数 (※)

第 8 章 ARM 汇编

- 8.1 ARM 架构概述 (※)
- 8.2 汇编语言基础 (※)
- 8.3 ARM 程序结构 (※)
- 8.4 伪操作 (※)

- 8.5 ARM 寄存器 (※)
- 8.6 ARM 工作模式 (※)
- 8.7 ARM 寻址方式 (※)
- 8.8 数据处理指令 (※)
- 8.9 分支指令
- 第 9 章 嵌入式环境搭建
 - 9.1 嵌入式系统与 Linux 内核
 - 9.2 内核版本类型 (※)
 - 9.3 内核编译工具与流程 (※)
 - 9.4 根文件系统 (rootfs) (※)
 - 9.5 QEMU 仿真环境
 - 9.6 交叉编译工具链 (※)
 - 9.7 Bootloader (※)

- 作业题
 - 题 1 Linux 基本命令操作
 - 题 2 Shell 脚本编程
 - 题 3 文件 IO 编程 (C语言)
 - 题 4 ARM 汇编编程

要点速背

0.1 高频要点

要点	核心内容
嵌入式系统定义	以应用为中心、以计算机技术为基础、采用 可剪裁 软硬件，适用于对功能、可靠性、成本、体积、功耗等有严格要求的 专用计算机系统
嵌入式7大特点	专用任务确定、高实时性、实时操作系统、高可靠性、有功耗约束、可用资源少、需专用工具和方法
嵌入式处理器4类	EMPU （基于通用CPU，外接存储）、 MCU/单片机 （整个系统集成一芯片）、 EDSP （数字信号处理）、 SoC （系统级集成）
交叉编译	在一个平台（宿主机）上编译，产出在 另一种体系结构 平台（目标机）上运行
本地编译	在当前平台编译，只能在 当前平台 运行
进程 vs 程序	程序：静态指令集合；进程：程序的一次 执行过程 （动态），是资源分配和调度基本单位
线程	进程内的执行流，是 CPU调度的最小单位 ，共享进程资源
任务	宽泛概念，可以是进程也可以是线程
并发 vs 并行	并发：同一时间段多个程序在 同一个CPU 交替执行（不一定同时）；并行：多个CPU 真正同时 执行
进程5种状态	<code>TASK_RUNNING</code> （运行/就绪）、 <code>TASK_INTERRUPTIBLE</code> （可中断等待）、 <code>TASK_UNINTERRUPTIBLE</code> （不可中断）、 <code>TASK_ZOMBIE</code> （僵死，PCB未释放）、 <code>TASK_STOPPED</code> （暂停）
fork() 返回值	父进程：返回子进程PID；子进程：返回0；出错：返回-1

要点	核心内容
fork vs vfork	fork: 拷贝 父进程数据段, 父子顺序不定; vfork: 共享 父进程数据段, 子进程先执行
exec函数族	替换当前进程代码和数据, PID不变 ; 成功则不返回; execl(path,arg0,...,NULL) / execlp (用PATH找) / execv(path,argv[])
wait/waitpid	wait()等待任意子进程结束, 防止僵死进程; waitpid(pid,&status,0)等待指定PID
管道特点	半双工 , 数据单向流动; 匿名管道只能用于 父子/兄弟进程
IPC六大机制	管道 (半双工)、 信号 (软件中断/异步通知)、 消息队列 (有格式消息)、 共享内存 (最快)、 信号量 (同步计数)、 套接字 (可跨网络)
信号量	非负整数, P操作 (申请资源/减1)、V操作 (释放资源/加1), 用于 进程同步与互斥
共享内存	最快 的IPC方式, 需配合信号量保证一致性; 步骤: shmget→shmat→访问→shmdt→shmctl删除
常见信号	SIGINT (Ctrl+C终止)、 SIGTERM (kill默认, 可捕获)、 SIGKILL (强制终止, 不可捕获/忽略)、 SIGALRM (alarm超时)
TCP vs UDP	TCP: 有连接、可靠、三次握手、字节流; UDP: 无连接、不可靠、速度快、适合实时音视频
OSI 七层模型	物理层→数据链路层→网络层 (IP) →传输层 (TCP/UDP) →会话层→表示层→应用层 (HTTP/FTP)
Socket服务器流程	socket()→bind()→listen()→循环 {accept()→read/recv()→write/send()→close(客户端fd)}
Socket客户端流程	socket()→connect()→write/send()→read/recv()→close()
字节序转换	发送前: htonl/htons (主机→网络大端序); 接收后: ntohl/ntohs (网络→主机)
ARM 架构	RISC精简指令集, 体积小、 低功耗 、低成本、高性能; 支持ARM(32位)/Thumb(16位)双指令集; 大量寄存器操作, 流水线技术
ARM 7种工作模式	用户 (User, 普通)、 快中断 (FIQ, 高优先级)、 普通中断 (IRQ)、 超级用户 (SVC, 复位/软中断)、 中止 (Abort, 存取异常)、 未定义 (Undef)、 系统 (System)
伪操作 AREA	定义代码/数据段; ENTRY指定入口; END表示源程序结尾; 伪操作不生成机器码
LR 寄存器	别名R14, 保存子程序返回地址 ; BL调用子程序, MOV PC, LR返回
特殊寄存器	PC(R15): 存放下一条指令地址; CPSR: 当前程序状态 (含NZCV标志位、模式位); SPSR: 异常时保存CPSR内容
Shell变量命名	字母/数字/下划线, 首字符不能是数字 , 不能有空格/标点/bash关键字; 等号两侧 不能有空格
Shell脚本执行	#!/bin/bash → chmod +x → ./script.sh ; 或直接 bash script.sh

要点	核心内容
Bootloader	操作系统运行前执行的程序， 初始化硬件 、加载内核、传递启动参数；常见： U-Boot （支持ARM/MIPS/PowerPC）
Bootloader 6 大功能	①初始化硬件 ②加载内核 ③设置启动参数 ④提供调试接口 ⑤支持固件升级 ⑥多重引导支持
rootfs	Linux根文件系统，包含/bin /boot /dev /etc /home /proc等基本目录； BusyBox 称为"Linux工具界的瑞士军刀"
内核版本类型	Prepatch (RC预发行)、 Mainline (主线，2~3月一版)、 Stable (稳定)、 Longterm (长期维护)
QEMU模式	全系统仿真 （qemu-system-xxx，运行完整OS）、 用户模式 （qemu-user，运行其他架构程序）、 虚拟化 （接近原生性能）

0.2 高频公式 / 关键结论

chmod 八进制权限

```

1 7 = rwx   6 = rw-   5 = r-x   4 = r--
2 chmod 777 a.sh    → 所有用户完全权限
3 chmod +x  a.sh    → 为所有用户添加执行权限
4 chmod u+x  a.sh    → 仅为文件所有者添加执行权限

```

Shell 变量规则（重要！）

```

1 var='value'      # 定义（等号两侧不能有空格）
2 echo $var        # 使用（$ 前缀）
3 readonly var     # 设为只读
4 unset var        # 删除变量

```

Shell 算术运算三种方式

```

1 val=`expr 2 + 2`      # expr（运算符两侧必须有空格，反引号包围）
2 ((c=a+b))             # 双括号
3 let c=a*b             # let 命令

```

Shell 条件判断与循环

```

1 # if语句（中括号两侧必须有空格！）
2 if [ $a -eq $b ]; then
3     命令
4 elif [ $a -gt $b ]; then
5     命令
6 else
7     命令
8 fi
9
10 # while循环
11 i=1
12 while (( i<=5 )); do

```

```
13     echo $i
14     i=`expr $i + 1`
15 done
16
17 # for循环（用列表，非C风格计数器）
18 for var in 1 2 3 4 5; do
19     echo $var
20 done
21
22 # case语句
23 case $变量 in
24     值1) 命令;;
25     值2) 命令;;
26     *) 默认;;
27 esac
```

Shell 脚本参数

```
1 $0 # 脚本名称
2 $1~$9 # 第1~9个参数（>=10用${10}）
3 $# # 参数个数
4 $* # 所有参数
5 $$ # 当前进程ID
6 $? # 上条命令退出状态（0=成功）
```

Shell 比较运算符

中括号写法	含义	双括号写法
-eq	相等	==
-ne	不等	!=
-gt	大于	>
-lt	小于	<
-ge	大于等于	>=
-le	小于等于	<=

标准文件描述符

描述符	名称	默认设备
0	stdin（标准输入）	键盘
1	stdout（标准输出）	显示器
2	stderr（标准错误）	显示器

POSIX 文件IO核心函数

```

1 | int fd = open(path, O_RDWR|O_CREAT|O_TRUNC, 0644); // 打开/创建
2 | read(fd, buf, count); // 读 (返回读取字节数, 0=文件尾, -1=出错)
3 | write(fd, buf, count); // 写
4 | lseek(fd, offset, SEEK_SET/SEEK_CUR/SEEK_END); // 定位
5 | close(fd); // 关闭 (失败返回-1)

```

C库函数IO核心函数

```

1 | FILE *fp = fopen(path, "r/w/a/rb/wb"); // 打开 (失败返回NULL)
2 | fread(ptr, size, n, fp); // 读n个size大小块
3 | fwrite(ptr, size, n, fp); // 写n个size大小块
4 | fgets(buf, size, fp); // 读一行 (到换行/文件尾/size)
5 | fseek(fp, offset, whence); // 定位
6 | fclose(fp); // 关闭

```

时间编程

```

1 | time_t t = time(NULL); // 获取日历时间 (1970-01-01至今的秒数)
2 | struct tm *p = gmtime(&t); // 转为年月日结构体
3 | printf("%d-%d-%d", 1900+p->tm_year, 1+p->tm_mon, p->tm_mday);
4 | // 注意: tm_year需+1900, tm_mon需+1

```

进程创建与等待

```

1 | pid_t pid = fork();
2 | if (pid == 0) { /* 子进程 */ }
3 | else if (pid > 0) { /* 父进程 */ wait(NULL); } // wait防僵死
4 |
5 | // exec替换进程 (PID不变, 成功不返回)
6 | execl("/bin/ls", "ls", "-al", NULL);
7 | execlp("ls", "ls", "-al", NULL); // 用PATH查找

```

Socket 服务器完整流程

```

1 | int srv = socket(AF_INET, SOCK_STREAM, 0);
2 | bind(srv, (struct sockaddr*)&addr, sizeof(addr));
3 | listen(srv, 5);
4 | while(1) {
5 |     int cli = accept(srv, NULL, NULL);
6 |     read(cli, buf, sizeof(buf));
7 |     write(cli, buf, strlen(buf));
8 |     close(cli);
9 | }

```

IP 地址分类

类	范围	最高位	用途
A类	1.0.0.0 ~ 126.0.0.0	0	大型网络
B类	128.0.0.0 ~ 191.255.0.0	10	中型网络

类	范围	最高位	用途
C类	192.0.0.0 ~ 223.255.255.0	110	小型网络
D类	224.0.0.0 ~ 239.255.255.255	1110	组播
E类	240.0.0.0 ~ 255.255.255.255	1111	保留

ARM 数据类型： 字节(Byte)=8b； 半字(Half Word)=16b(2B)； 字(Word)=32b(4B)

ARM 指令格式： <opcode>{<cond>}{S} <Rd>, <Rn>, <op2>

- {S}：结果影响CPSR标志位
- {cond}：条件码（EQ/NE/GT/LT/GE/LE等）

ARM 常用指令速记

1	MOV R0, #10	； R0 = 10（传送立即数）
2	MOV PC, LR	； 子程序返回
3	ADD R2, R0, R1	； R2 = R0 + R1
4	SUB R0, R0, #1	； R0 = R0 - 1
5	ORR R0, R0, #0x0F	； R0低4位置1（或）
6	AND R0, R0, #0x0F	； R0只保留低4位（与）
7	BIC R0, R0, #0x0F	； R0低4位清零（位清除）
8	EOR R1, R1, #0x0F	； R1低4位取反（异或）
9	CMP R0, R1	； R0-R1更新标志位（不保存结果）
10	LDR R0, [R1]	； R0 = 内存[R1]（加载）
11	STR R0, [R1]	； 内存[R1] = R0（存储）

ARM 寻址方式速记

寻址方式	示例	特点
立即数寻址	MOV R0, #5	操作数为常数（#）
寄存器寻址	MOV R0, R1	操作数全为寄存器
寄存器间接寻址	LDR R0, [R1]	[寄存器]取其地址值
基址变址寻址	LDR R0, [R1, #4]	基址+偏移
相对寻址	BEQ label	PC为基址+偏移
多寄存器寻址	LDMIA R0, {R1-R5}	一次加载多个寄存器

BL 子程序调用流程

1	BL SUBNAME	； 调用（LR = 下一条指令地址）
2	； ... 主程序 ...	
3	SUBNAME	
4	； ... 子程序内容 ...	
5	MOV PC, LR	； 返回

信号量 P/V 操作

- P操作（wait/down）：信号量>0则减1并继续；=0则阻塞等待（请求资源）
- V操---m-arm -M vexpress-a9 -m 512M -kernel zImage -nographic

用户模式运行ARM程序

```
qemu-arm -L /usr/arm-linux-gnueabihf ./prog
```

1			
2	---		
3			
4			
5	## 0.1 高频要点		
6			
7	要点	核心内容	
8	-----	-----	
9	嵌入式系统定义	以应用为中心、以计算机技术为基础、采用**可剪裁**软硬件，适用于对功能、可靠性、成本、体积、功耗等有严格要求的**专用计算机系统**	
10	嵌入式7大特点	专用任务确定、高实时性、实时操作系统、高可靠性、有功耗约束、可用资源少、需专用工具和方法	
11	交叉编译	在一个平台（宿主机）上编译，产出在**另一种体系结构**平台（目标机）上运行	
12	本地编译	在当前平台编译，只能在**当前平台**运行	
13	进程 vs 程序	程序：静态指令集合；进程：程序的一次**执行过程**（动态），是资源分配和调度基本单位	
14	线程	进程内的执行流，是**CPU调度的最小单位**，共享进程资源	
15	任务	宽泛概念，可以是进程也可以是线程	
16	并发 vs 并行	并发：同一时间段多个程序在**同一个CPU**交替执行（不一定同时）；并行：多个CPU**真正同时**执行	
17	fork() 返回值	父进程：返回子进程PID；子进程：返回0；出错：返回-1	
18	管道特点	**半双工**，数据单向流动；匿名管道只能用于**父子/兄弟**进程	
19	信号量	非负整数，P操作（申请资源/减1）、V操作（释放资源/加1），用于**进程同步与互斥**	
20	共享内存	**最快**的IPC方式，需配合信号量/互斥锁保证数据一致性	
21	TCP vs UDP	TCP：有连接、可靠、三次握手；UDP：无连接、不可靠、速度快	
22	ARM 架构	RISC精简指令集，体积小、**低功耗**、低成本、高性能；支持ARM(32位)/Thumb(16位)双指令集	
23	伪操作 AREA	定义代码/数据段；ENTRY指定入口；END表示源程序结尾；**伪操作不生成机器码**	
24	LR 寄存器	别名R14，**保存子程序返回地址**；BL调用子程序，MOV PC, LR返回	
25	特殊寄存器	PC(R15)：存放下一条指令地址；CPSR：当前程序状态；SPSR：保存CPSR内容	
26	Bootloader	操作系统运行前执行的程序，**初始化硬件**、加载内核、传递启动参数；常见：U-Boot	


```
27 | rootfs          | Linux根文件系统，包含/bin /boot /dev /etc /home /proc等基本目
    | 录
28
29 ## 0.2 高频公式 / 关键结论
30
31 **chmod 八进制权限**
```

7 = rwx 6 = rw- 5 = r-x 4 = r--
chmod 777 a.sh → 所有用户完全权限
chmod +x a.sh → 为所有用户添加执行权限
chmod u+x a.sh → 仅为文件所有者添加执行权限

```
1
2 **Shell 变量规则**（重要！）
3 ``bash
4 var='value'      # 定义（等号两侧不能有空格）
5 echo $var        # 使用（$ 前缀）
6 readonly var     # 设为只读
7 unset var        # 删除变量
```

Shell 算术运算三种方式

```
1 val=`expr 2 + 2`      # expr（运算符两侧必须有空格，反引号包围）
2 ((c=a+b))             # 双括号
3 let c=a*b             # let 命令
```

标准文件描述符

描述符	名称	默认设备
0	stdin（标准输入）	键盘
1	stdout（标准输出）	显示器
2	stderr（标准错误）	显示器

IP 地址分类

类	范围	最高位	用途
A类	1.0.0.0 ~ 126.0.0.0	0	大型网络
B类	128.0.0.0 ~ 191.255.0.0	10	中型网络
C类	192.0.0.0 ~ 223.255.255.0	110	小型网络
D类	224.0.0.0 ~ 239.255.255.255	1110	组播
E类	240.0.0.0 ~ 255.255.255.255	1111	保留

ARM 数据类型：字节(Byte)=8b；半字(Half Word)=16b(2B)；字(Word)=32b(4B)

ARM 指令格式：<opcode>{<cond>}{S} <Rd>，<Rn>，<op2>

- {S}：结果影响CPSR标志位

- `{cond}`：条件码 (EQ/NE/GT/LT/GE/LE等)

BL 子程序调用流程

```
1 BL SUBNAME ; 调用 (LR = 下一条指令地址)
2 ; ... 主程序 ...
3 SUBNAME
4 ; ... 子程序内容 ...
5 MOV PC, LR ; 返回
```

信号量 P/V 操作

- P操作 (wait/down)：信号量>0则减1并继续；=0则阻塞等待 (请求资源)
- V操作 (signal/up)：信号量加1，通知其他等待进程 (释放资源)

0.3 易混淆点

容易混淆的概念	区分方法
进程 vs 线程	进程是 资源分配 单位 (独立地址空间)；线程是 CPU调度 单位 (共享进程资源)
并发 vs 并行	并发：单CPU轮流，不一定同时；并行：多CPU，一定同时
匿名管道 vs FIFO	匿名管道只限父子/兄弟进程；FIFO可用于 任意 两进程，在文件系统中持久存在
fork() vs vfork()	fork拷贝父进程数据段；vfork 共享 父进程数据段，子进程先执行
信号(signal) vs 信号量 (semaphore)	signal：异步通知事件 (如SIGINT/SIGKILL)；semaphore：同步计数器，控制资源访问
POSIX文件IO vs C库函数 IO	POSIX：open/read/write/close，返回文件描述符(int)；C库：fopen/fread/fwrite/fclose，返回FILE指针
伪操作 vs 伪指令	伪操作：汇编阶段起作用， 不生成 机器码；伪指令：被替换成真实ARM/Thumb指令
TCP vs UDP	TCP：有连接、可靠、三次握手、字节流；UDP：无连接、不可靠、速度快、适合实时应用
> vs >> 重定向	>：覆盖写入；>>：追加写入
cat vs more	cat：一次显示全部内容；more：支持翻页 (空格翻页，q退出)
单引号 vs 双引号	双引号： 能识别 变量，能转义 (\n \t)；单引号：原样输出，不识别变量
CMP vs SUBS	CMP不保存运算结果，只更新标志位；SUBS保存结果且更新标志位
TST vs ANDS	TST不保存结果，只更新标志位 (用于位测试)；ANDS保存结果
CMN vs ADDS	CMN：Rn+op2，不保存结果 (用于负数比较)；ADDS：保存结果

第 1 章 嵌入式系统绪论

1.3 嵌入式系统结构

四大组成部分

- 1. 嵌入式微处理器（处理核心）
- 2. 外围硬件设备（总线、存储器、I/O、通讯设备）
- 3. 嵌入式操作系统（可选，如Linux、VxWorks、Android）
- 4. 用户应用软件系统

嵌入式处理器分类

类型	特点	代表
嵌入式微处理器(EMPUP)	基于通用CPU，外接ROM/RAM/总线	ARM、MIPS、PowerPC
嵌入式微控制器(MCU/单片机)	整个计算机系统集成在一块芯片	8051、STM32
嵌入式DSP处理器(EDSP)	专用于数字信号处理	TI TMS320
嵌入式片上系统(SoC)	系统级集成	手机SoC

1.4 嵌入式操作系统

常见嵌入式OS：嵌入式Linux、Windows CE、VxWorks、QNX、Android、iOS、uC/OS-II

RTOS性能指标：内核大小几K几百K；时间片1ms；实时任务响应时间20~40微秒

Linux 发行版组成：Linux内核 + GNU工具集和库 + 图形界面 + 应用软件 + 软件包管理系统 (APT/Yum)

常用Linux发行版

- 桌面：Ubuntu、Fedora、Linux Mint
- 服务器：CentOS、Debian、RHEL
- 移动/嵌入式：Android（基于Linux内核）、Chrome OS

第 2 章 Linux 基本操作

2.1 Linux 目录结构

- Linux只有一个根目录 / （Windows有多个盘符）
- 目录结构是树型结构，层级关系用 / 表示
- Windows用 \ 表示层级，Linux用 / 表示层级

命令基础格式： `command [-options] [parameter]`

- [] 表示可选，parameter 为指向目标的参数

2.2 常用命令 (※)

目录和文件操作命令

命令	功能	常用参数
<code>ls</code>	列出目录内容	<code>-a</code> 显示隐藏文件; <code>-l</code> 详细信息; <code>-h</code> 文件大小友好显示 (需配合 <code>-l</code>)
<code>pwd</code>	显示当前工作目录 (Print Work Directory)	—
<code>cd 路径</code>	切换目录 (Change Directory) ; 不加参数返回HOME	<code>cd ~</code> 回HOME; <code>cd ..</code> 上级; <code>cd -</code> 回上一个打开的目录
<code>mkdir</code>	创建目录 (Make Directory)	<code>-p</code> 递归创建多级目录
<code>touch</code>	创建文件	—
<code>cat</code>	显示文件内容 (一次性全部显示)	—
<code>more</code>	分页显示文件内容	空格翻页, <code>q</code> 退出
<code>cp</code>	复制文件/文件夹	<code>-r</code> 递归 (复制文件夹必须用)
<code>mv</code>	移动/重命名文件	—
<code>rm</code>	删除文件/文件夹	<code>-r</code> 递归; <code>-f</code> 强制删除 (不提示) ; 慎用 <code>rm -rf /</code>
<code>echo</code>	输出文本内容	复杂内容用 <code>" "</code> 包围
<code>tail</code>	查看文件尾部内容	<code>-f</code> 持续跟踪; <code>-num</code> 查看尾部N行 (默认10行)

查找命令

命令	功能	示例
<code>which 命令</code>	查找命令程序文件位置	<code>which ls</code>
<code>find / -name "file"</code>	按文件名查找	支持通配符 <code>*</code>
<code>find / -size +100M</code>	按文件大小查找	<code>+</code> 大于, <code>-</code> 小于; 单位k/M/G
<code>grep</code>	从文件中过滤关键字行	<code>-n</code> 显示行号
<code>wc</code>	统计文件内容	<code>-l</code> 行数; <code>-w</code> 单词数; <code>-c</code> 字节数; <code>-m</code> 字符数

示例

```
1 ls -la # 显示详细信息（含隐藏文件）
2 ls -lh # 以易读方式显示文件大小
3 find / -name "*test*" # 查找所有包含test的文件
4 grep -n "power" a.txt # 过滤含power的行并显示行号
5 cat a.txt | grep power # 管道符过滤
6 cat a.txt | grep power | wc -l # 统计匹配行数
```

2.3 路径与特殊符号 (※)

路径类型

- **绝对路径**：以根目录 / 开头（如 /home/user/test）
- **相对路径**：以当前目录为起点，不以 / 开头（如 Desktop、./Desktop）

特殊路径符

符号	含义	示例
.	当前目录	cd ./Desktop 等同于 cd Desktop
..	上一级目录	cd ../.. 跳两级
~	HOME目录	cd ~ 跳到HOME; cd ~/Desktop

2.4 权限管理 (※)

权限字符串格式 (共10位)

```
1 drwxr-xr-x
2 |  |  |  |  |  |
3 | Owner Group Others
4 |—— 文件类型：d=目录，l=链接，- =普通文件
5 r=读，w=写，x=执行，- =无此权限
```

chmod 命令 (※高频)

```
1 chmod +x a.sh # 为所有用户添加执行权限
2 chmod u+x a.sh # 仅为文件所有者添加执行权限
3 chmod 777 a.sh # 所有用户完全权限（rwxrwxrwx）
4 chmod 644 a.sh # Owner读写，其余只读（rw-r--r--）
5 chmod 755 a.sh # Owner完全权限，其余读和执行
```

权限数字对照 (※)

数字	权限	说明
7	rwx	读+写+执行
6	rw-	读+写
5	r-x	读+执行
4	r--	只读

数字	权限	说明
0	---	无权限

chmod权限位分组：从左往右，第1位特殊权限，第2位拥有者，第3位group，第4位other；6=RW-
sudo / su： `sudo` 临时以管理员权限执行单条命令； `su` 切换为root用户身份

2.5 管道、重定向与vi编辑器

管道符 |

- 将左边命令的**输出**作为右边命令的**输入**
- 可嵌套使用：`cat file | grep key1 | grep key2`

重定向符

- `>`：覆盖写入到文件
- `>>`：追加写入到文件
- 例：`echo "Hello" > h1.txt`；`echo "world" >> h1.txt`

反引号 `

```
1 | echo `pwd`           # 输出当前工作目录路径（而非字符串pwd）
2 | echo "Current: `pwd`" > work.txt
```

vi/vim 编辑器三种模式

- **命令模式**（默认进入）：所有按键作为命令，不能直接输入文字
- **输入模式**：按 `i` 进入，可自由编辑文字，`ESC` 退出
- **底线命令模式**：命令模式下按 `:` 进入，用于保存/退出

底线命令： `:wq` 保存并退出；`:q!` 强制不保存退出；`:w` 只保存

文件压缩解压 (※)

命令	功能
<code>tar -xvf file.tar.xz</code>	解压tar.xz
<code>tar -xjvf file.tar.bz2 -C 目录</code>	解压bz2到指定目录
<code>tar -czvf archive.tar.gz 目录/</code>	将目录打包压缩为gzip文件
<code>tar -xzvf archive.tar.gz</code>	解压gzip文件
<code>unzip filename.zip</code>	解压zip文件
<code>unzip -d 目录 filename.zip</code>	解压到指定目录

Linux 关机/重启： `shutdown now` / `poweroff` / `halt -p`；重启：`reboot` / `shutdown -r`

包管理工具 (※)

发行版	工具	安装示例
Ubuntu/Debian	APT	<code>sudo apt-get install build-essential</code>
CentOS	YUM	<code>yum install gcc</code>

用户管理命令： `useradd` 用户名（添加）； `passwd` 用户名（修改密码）； `userdel` 用户名（删除）； `su` 用户名（切换用户，`exit` 退回）； `cat /etc/passwd`（查看所有用户）

第 3 章 Shell 编程

3.1 Shell 概述

Shell 定义

- 用C语言编写，是用户使用Linux的**桥梁**（命令语言+程序设计语言）
- 为用户提供访问操作系统内核服务的界面
- Linux默认Shell： `/bin/bash`（Bourne Again Shell）

Shell 脚本使用流程（※）

```
1 touch test.sh           # 1. 创建脚本文件
2 vi test.sh              # 2. 编写脚本（首行必须是 #!/bin/bash）
3 chmod +x test.sh        # 3. 赋予执行权限
4 ./test.sh               # 4. 执行（必须加 ./ 前缀）
```

3.2 变量（※）

变量定义与使用

```
1 #!/bin/bash
2 words='Hello world'      # 定义（等号两侧不能有空格）
3 echo $words              # 使用（$ 前缀）
4 echo "$words"            # 双引号：识别变量
5 echo '$words'            # 单引号：原样输出 $words
6
7 # 将命令执行结果赋值给变量（使用反引号）
8 dt=`date +%F %T`
9 echo $dt
```

变量命名规则（※）

- 只能用英文字母、数字、下划线，**首字符不能是数字**
- 中间不能有空格
- 不能使用标点符号
- 不能使用bash关键字（ls、echo、pwd、find等）

合法变量名： `var`、`my_var`、`var2`

非法变量名： `?var`（有?）、`user*name`（有*）、`echo`（关键字）、`2var`（数字开头）

变量操作

```
1 readonly a      # 设为只读变量（不能再赋值）
2 unset b         # 删除变量
3 read -p '提示: ' var # 读取用户输入
```

3.3 算术运算 (※)

expr 方式（表达式和运算符之间**必须有空格**，使用**反引号**）

```
1 val=`expr 2 + 2`      # 正确
2 val=`expr 2+2`        # 错误！没有空格
3 val=`expr $a \* $b`    # 乘法需转义 *
4 val=`expr $b / $a`     # 除法
5 val=`expr $b % $a`     # 取余
```

双括号 (()) 方式

```
1 a=5; b=3
2 ((c=a+b))             # 加
3 ((c=a*b))             # 乘
4 echo $c
```

let 命令

```
1 a=3; b=2
2 let c=a*b
3 echo $c
```

算术运算符总结

运算符	说明	注意
<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>	基本四则+取余	<code>expr</code> 中 <code>*</code> 需转义为 <code>*</code>
<code>=</code>	赋值	<code>a=\$b</code>
<code>==</code> <code>!=</code>	比较	用于条件判断

3.4 条件判断 (※)

比较运算符（中括号写法）

运算符	含义	双括号/let写法
<code>-eq</code>	相等	<code>==</code>
<code>-ne</code>	不等	<code>!=</code>
<code>-gt</code>	大于	<code>></code>
<code>-lt</code>	小于	<code><</code>
<code>-ge</code>	大于等于	<code>>=</code>

运算符	含义	双括号/let写法
<code>-le</code>	小于等于	<code><=</code>

注意：条件表达式要放在方括号之间，且**两侧必须有空格**：`[ $a -eq $b ]`（正确），`[$a==$b]`（错误）

逻辑运算符

运算符	说明
<code>!</code>	非运算
<code>-o</code>	或运算（有一个true则true）
<code>-a</code>	与运算（都为true才true）

字符串运算符：`=` 相等；`!=` 不等；`-z` 长度为0；`-n` 长度不为0；`str` 非空

文件测试运算符（※常用）

运算符	含义
<code>-e file</code>	文件/目录是否存在
<code>-f file</code>	是否是普通文件
<code>-d file</code>	是否是目录
<code>-r/-w/-x file</code>	是否可读/可写/可执行
<code>-s file</code>	文件不为空（大小>0）

3.5 控制流

if 语句

```
1  if [ 条件 ]; then
2      命令
3  elif [ 条件 ]; then
4      命令
5  else
6      命令
7  fi
8
9  # 双括号写法
10 if (( a == 2 )); then
11     echo 'a equals 2'
12 fi
```

case 语句

```

1 case $变量 in
2     值1)
3         命令;;
4     值2)
5         命令;;
6     *)
7         默认命令;;
8 esac

```

while 循环

```

1 while (( 条件 )); do
2     命令
3 done
4
5 # 死循环 (Shell中非0为真)
6 while ((1)); do
7     echo "running"
8     sleep 1
9 done
10 # Ctrl+C 强制结束

```

for 循环 (用列表而非计数器控制, 与C语言不同)

```

1 for var in 值1 值2 值3; do
2     命令
3 done
4
5 # 示例: 输出1-5
6 i=1
7 while (( $i<=5 )); do
8     echo $i
9     i=`expr $i + 1`
10 done

```

break / continue: `break` 跳出所有循环; `continue` 跳出当前次循环

3.6 函数与参数传递

函数定义与调用

```

1 funname() {
2     action
3     [return int]
4 }
5 funname    # 调用
6
7 # 带return的函数
8 add() {
9     return $(( $1 + $2 ))
10 }
11 add 3 5
12 echo "Result: $?"    # $? 获取函数返回值

```

脚本参数传递 (※)

符号	说明
<code>\$1 ~ \$9</code>	第1~9个参数 (≥10时用 <code>\${10}</code>)
<code>\$0</code>	脚本名称
<code>\$#</code>	参数个数
<code>\$*</code>	所有参数
<code>\$\$</code>	当前进程ID
<code>\$?</code>	上一条命令的退出状态 (0=成功)

数组

```
1 arr=(A B C D)           # 定义
2 echo ${arr[0]}          # 访问第一个元素
3 echo ${arr[*]}          # 所有元素
4 echo ${#arr[*]}         # 数组长度
5 arr[0]=value0          # 单独赋值
```

第 4 章 文件 IO 编程

4.1 文件流与标准流

文件流 (File Stream)

- C语言将文件看作字节序列，分为**文本流**和**二进制流**
- 3个预定义标准流：

名称	说明	文件描述符
stdin	标准输入 (默认：键盘)	0
stdout	标准输出 (默认：显示器)	1
stderr	标准错误 (默认：显示器)	2

4.2 POSIX 文件 IO (系统调用) (※)

POSIX (Portable Operating System Interface)：IEEE制定的标准 (IEEE 1003)，确保程序在类UNIX系统间的可移植性

文件描述符：非负整数，0/1/2已预留，操作失败返回 `-1` 并设置 `errno`

核心函数

函数	功能	返回值
<code>open(path, flags [,mode])</code>	打开文件	文件描述符(int)，出错-1

函数	功能	返回值
<code>creat(path, mode)</code>	创建文件	文件描述符, 出错-1
<code>close(fd)</code>	关闭文件描述符	成功0, 出错-1
<code>write(fd, buf, count)</code>	写入数据	写入字节数, 出错-1
<code>read(fd, buf, count)</code>	读取数据	读取字节数, 到末尾0, 出错-1
<code>lseek(fd, offset, whence)</code>	移动读写位置	新的偏移量
<code>fcntl(fd, cmd, ...)</code>	操作文件描述符特性	—
<code>flock(fd, operation)</code>	文件锁定/解锁	成功0, 出错-1

open 的 flags 参数

- `O_RDONLY`：只读；`O_WRONLY`：只写；`O_RDWR`：读写（三选一）
- `O_CREAT`：不存在则创建；`O_TRUNC`：清空原内容；`O_APPEND`：追加

creat() 等价写法： `open(path, O_CREAT|O_WRONLY|O_TRUNC, mode)`

lseek 第三参数

- `SEEK_SET`：从文件开头偏移
- `SEEK_CUR`：从当前位置偏移
- `SEEK_END`：从文件末尾偏移

flock 操作类型

- `LOCK_EX`：排他锁（阻止其他进程读写）
- `LOCK_SH`：共享锁（允许读，阻止写）
- `LOCK_UN`：解锁
- `LOCK_NB`：非阻塞

4.3 C 语言库函数 IO (※)

C语言库函数IO：跨平台，独立于操作系统，封装底层系统调用

函数	功能
<code>fopen(path, mode)</code>	打开文件, 返回 <code>FILE*</code>
<code>fclose(fp)</code>	关闭文件
<code>fwrite(ptr, size, n, fp)</code>	写入 (n个size大小的数据块)
<code>fread(ptr, size, n, fp)</code>	读取 (n个size大小的数据块)
<code>fgetc(fp)</code>	读取单个字符 (到文件尾返回EOF)
<code>fgets(s, size, fp)</code>	读取一行 (到换行符/文件尾/size为止)
<code>fputc(c, fp)</code>	写入单个字符

函数	功能
<code>fputs(str, fp)</code>	写入字符串
<code>fscanf(fp, fmt, ...)</code>	格式化读取
<code>fprintf(fp, fmt, ...)</code>	格式化写入（写到文件，类似printf）
<code>fseek(fp, offset, whence)</code>	移动文件指针

fopen 模式

- `r`：只读；`w`：只写（清空或创建）；`a`：追加写
- `rb/wb`：二进制读/写

字符串结束符 `\0`：C语言字符串以 `\0`（ASCII值0）结尾，`fprintf` 遇到 `\0` 停止输出

POSIX vs C库函数对比

维度	POSIX	C库函数
返回值	文件描述符(int)	FILE指针(FILE*)
错误标志	返回-1, 设errno	返回NULL
可移植性	类UNIX系统	跨平台
层次	底层系统调用	高层封装

4.4 时间编程 (※)

两个核心概念

概念	说明
UTC（世界协调时）	基于原子时钟的国际时间标准
日历时间	从1970年1月1日0时0分0秒算起到现在的秒数

时间相关函数

函数	功能
<code>time_t time(time_t *t)</code>	返回日历时间（秒数，约17亿秒）
<code>struct tm *gmtime(const time_t *timep)</code>	将秒数转换为年月日时分秒结构体
<code>time_t mktime(struct tm *timeptr)</code>	将tm结构体转换回秒数

tm 结构体关键字段（注意：年份需+1900，月份需+1）

```

1 struct tm {
2     int tm_sec;    // 秒
3     int tm_min;    // 分
4     int tm_hour;   // 时
5     int tm_mday;   // 当月第几日
6     int tm_mon;    // 月（0-11，实际月份需+1）
7     int tm_year;   // 年（从1900年起算，实际年份需+1900）
8     int tm_wday;   // 星期（0=周日）
9     int tm_yday;   // 当年第几天
10    int tm_isdst;   // 是否夏令时
11 };

```

打印年月日示例 (※)

```

1 #include <time.h>
2 #include <stdio.h>
3 void main() {
4     time_t timep;
5     struct tm *p;
6     time(&timep);
7     p = gmtime(&timep);
8     printf("%d-%d-%d\n", (1900+p->tm_year), (1+p->tm_mon), p->tm_mday);
9 }

```

date 命令：功能强大的Linux日期时间工具，`date +%F %T` 输出年月日+时分秒

GCC 编译：`gcc a.c -o a`（将C源码编译为可执行程序）

第 5 章 进程

5.1 基本概念 (※)

核心概念对比

概念	定义	单位
程序	静态的，存放于磁盘的指令集合	—
进程	程序的一次 执行过程 （动态），有生命周期	资源分配 基本单位
线程	进程内的执行流	CPU调度 最小单位，共享进程资源
任务	宽泛概念，可以是进程也可以是线程	—

并发 vs 并行 (※)

- **并发：**同一时间段内，多个程序在**同一个CPU**交替执行（时分复用）
 - 例：打字和听音乐在同一台电脑上都在进行 → 并发
- **并行：**系统有**多个CPU**时，多个程序**真正同时**执行
 - 例：一边打电话一边吃饭 → 并行

5.2 进程特性与状态

进程四大特性：动态性（有生命周期）、并发性、独立性（独立地址空间）、制约性

进程五种状态（Linux内核）（※）

状态	说明
<code>TASK_RUNNING</code>	正在运行或在就绪队列中等待执行
<code>TASK_INTERRUPTIBLE</code>	可中断的等待（可被信号唤醒）
<code>TASK_UNINTERRUPTIBLE</code>	不可中断的等待（不能被信号唤醒）
<code>TASK_ZOMBIE</code>	僵死状态： 进程结束但PCB（进程控制块）未释放
<code>TASK_STOPPED</code>	被暂停，需其他进程信号才能唤醒

状态转换

- 就绪→执行：CPU空闲时调度
- 执行→就绪：时间片用完
- 执行→阻塞：等待I/O或信号
- 阻塞→就绪：等待的事件发生

5.3 进程组成

进程控制块（PCB / `task_struct`）：内核通过双向循环链表（任务队列）管理所有进程

PCB 包含：PID（进程唯一标识符）、进程状态、程序计数器、寄存器组、内存信息（数据段/堆栈）、打开的文件、挂起的信号等

PID/PPID

- PID：正整数，唯一标识进程（类型 `pid_t`，实为 `int`）
- PPID：父进程ID；每个进程（除init/systemd外）都有父进程

5.4 进程管理命令

命令	功能
<code>ps</code>	查看当前运行的进程状态（Process Status）
<code>top</code>	动态实时显示进程信息（类似Windows任务管理器）
<code>kill PID</code>	向进程发送信号（默认终止）；用法： <code>kill 1405</code>
<code>nice</code>	按指定优先级运行
<code>renice</code>	改变运行中进程的优先级
<code>crontab</code>	提交和管理周期性任务
<code>bg</code>	将挂起的进程放到后台执行

命令	功能
<code>nohup ./a.sh &</code>	后台运行脚本（不挂起，配合 <code>tail -f nohup.out</code> 查看输出）

5.5 进程控制编程（※）

获取 PID

```
1 #include <unistd.h>
2 pid_t pid = getpid();    // 获取当前进程PID
3 pid_t ppid = getppid();  // 获取父进程PID
```

创建子进程 `fork()`（※高频）

```
1 #include <sys/types.h>
2 #include <unistd.h>
3 pid_t pid = fork();
4 if (pid < 0)           // 出错
5     printf("error!");
6 else if (pid == 0)     // 子进程（fork返回0）
7     printf("Child: PID=%d\n", getpid());
8 else                  // 父进程（fork返回子进程PID）
9     printf("Parent: PID=%d\n", getpid());
```

`fork()` vs `vfork()`（※）

特性	<code>fork()</code>	<code>vfork()</code>
数据段	拷贝父进程数据段	共享父进程数据段
执行顺序	父子进程顺序不确定	子进程先执行，子进程exit/exec后父进程才调度
地址空间	独立	在父进程的地址空间中运行
返回值	同fork	成功：父返回子PID，子返回0；失败：返回-1，errno记录原因

exec 函数族（※）：替换当前进程的代码和数据，加载并运行另一个程序（PID不变，常用于子进程执行新程序）

- 成功则**不返回**（当前程序被完全替换）；失败返回 -1
- `execl(path, arg0, arg1, ..., NULL)`：参数以可变参数列表传递（最后必须是NULL）
- `execle(file, arg0, ..., NULL)`：从PATH环境变量查找程序
- `execv(path, argv[])`：参数以**数组指针**形式传递
- 示例：`execl("/bin/ls", "ls", "-al", "/etc/passwd", (char*)0)` 相当于执行 `ls -al /etc/passwd`

system() 函数：调用 `fork()` 产生子进程，由子进程执行命令字符串，执行完后返回原进程

- `int system(const char *string);`

- 调用 `/bin/sh -c string` 执行命令
- 失败返回127 (shell调用失败) 或-1 (其他错误) , 成功返回命令退出状态

wait() / waitpid() 函数 (※) : 父进程等待子进程结束, 避免僵死进程

```
1 #include <sys/types.h>
2 #include <sys/wait.h>
3 pid_t wait(int *status);           // 等待任意子进程结束
4 pid_t waitpid(pid_t pid, int *status, int options); // 等待指定子进程
```

- 成功返回子进程PID, 出错返回-1
- `waitpid(pid=0)`: 等待同组进程; `pid=-1`: 等待任意子进程; `pid>0`: 等待指定PID

exit() 函数: 终止当前进程, 缓冲区数据自动写回, 关闭未关闭文件

- `void exit(int status);`
- `status=0`: 正常退出; 非0: 异常退出
- `exit`后的语句不会被执行

5.6 任务分类

按到达时间可预测性

- **周期任务:** 固定时间间隔执行 (如每100ms采样飞行器状态)
- **非周期任务:** 执行间隔不确定
 - 偶发性任务 (sporadic) : 有最小到达间隔限制
 - 非周期性任务 (aperiodic) : 无到达时间限制

按重要程度

- **关键任务:** 必须及时执行, 否则灾难性后果 (如飞行器生命支持系统)
- **非关键任务:** 不及时执行不会产生严重后果

第 6 章 进程间通信

6.1 IPC 概述

IPC (Inter-Process Communication) : 多个进程之间数据交换和信息共享的方式

Linux 六大 IPC 机制 (※)

机制	特点	用途
管道 (Pipe)	半双工, 只能用于有亲缘关系进程	简单数据流传输
信号 (Signal)	异步通知机制, 通知进程发生了某事件	事件通知
消息队列 (Message)	有格式消息, 异步通信	结构化数据传递
共享内存	最快的IPC方式, 直接访问同一内存	大量数据频繁交换
信号量 (Semaphore)	同步计数器, 控制共享资源访问	进程同步/互斥

机制	特点	用途
套接字 (Socket)	可用于不同主机间的网络通信	网络通信

6.2 管道 (Pipe) (※)

匿名管道特点

- **半双工**：数据只能单向流动（双向通信需两个管道）
- 只能用于**有亲缘关系**的进程（父子或兄弟进程）
- 管道对进程而言就是一个文件

创建匿名管道

```
1 #include <unistd.h>
2 int pipe(int filedes[2]);
3 // filedes[0]: 读取端; filedes[1]: 写入端
4 // 成功返回0, 失败返回-1
```

管道 IO 函数

```
1 // 建立管道（内部fork子进程并通过sh执行command）
2 FILE* popen(const char *command, const char *type);
3 // type: "r"从子进程读取, "w"向子进程写入
4 // 返回文件指针, 失败返回NULL
5
6 int pclose(FILE *stream); // 关闭管道, 返回子进程结束状态
```

示例

```
1 FILE *fp = popen("cat /etc/passwd", "r");
2 fgets(buffer, sizeof(buffer), fp); // fgets读到换行符为止
3 printf("%s", buffer);
4 pclose(fp);
```

6.3 FIFO (命名管道) (※)

FIFO 与匿名管道的区别

特性	匿名管道	FIFO (命名管道)
适用进程	仅父子/兄弟进程	任意两个进程
生命周期	与进程相关, 进程结束则关闭	存在于文件系统, 直到被删除
文件系统	无节点	在文件系统中存在节点（不存储数据）

创建命名管道

```
1 #include <sys/stat.h>
2 int mkfifo(const char *pathname, mode_t mode);
3 // 成功返回0，失败返回-1
4 // 创建后用普通文件读写方式存取
```

6.4 消息队列 (※)

消息队列特点

- 允许进程间传递**有格式**的消息（克服管道只能传无格式字节流的缺点）
- 提供**异步通信**能力
- 每条消息：**消息头**（类型、优先级等）+ **消息体**（实际数据）

核心函数

函数	功能
<code>key_t ftok(path, proj)</code>	生成唯一键值key（用于标识特定IPC资源）
<code>int msgget(key, msgflg)</code>	创建/打开消息队列（ <code>IPC_CREAT</code> 创建）
<code>int msgsnd(msqid, msgp, msgsz, msgflg)</code>	发送消息（ <code>IPC_NOWAIT</code> ：不阻塞）
<code>int msgrcv(msqid, msgp, msgsz, msgtyp, msgflg)</code>	接收消息（读取后队列中该消息被删除）
<code>int msgctl(msqid, cmd, buf)</code>	控制（ <code>IPC_RMID</code> 删除； <code>IPC_STAT</code> 获取信息）

消息结构体格式

```
1 struct msgp {
2     long mtype;    // 消息类型（必须 >0）
3     // 消息正文数据...
4 };
```

6.5 信号 (Signal) (※)

信号定义：软件中断机制，用于进程间传递事件通知

常见信号

信号	触发条件	默认行为
<code>SIGINT</code>	用户按 Ctrl+C	终止进程
<code>SIGTERM</code>	<code>kill</code> 命令默认发送	请求进程优雅退出（可被捕获）
<code>SIGKILL</code>	强制终止	立即停止， 无法被捕获/忽略
<code>SIGALRM</code>	<code>alarm()</code> 超时	终止进程

信号相关函数

函数	功能
<code>alarm(seconds)</code>	设置定时器，seconds秒后发送SIGALRM
<code>kill(pid, sig)</code>	向进程发送信号（pid=0：同组；pid=-1：广播）
<code>pause()</code>	暂停进程，直到被信号中断
<code>sleep(seconds)</code>	暂停指定秒数或被信号中断
<code>sigaction(signum, act, oldact)</code>	查询或设置信号处理方式
<code>sigprocmask(how, set, oldset)</code>	设置信号遮罩（阻塞特定信号）

信号集操作函数

函数	功能
<code>sigemptyset(&set)</code>	初始化信号集为空
<code>sigfillset(&set)</code>	将所有信号加入信号集
<code>sigaddset(&set, signum)</code>	将指定信号加入信号集
<code>sigdelset(&set, signum)</code>	从信号集删除指定信号
<code>sigismember(&set, signum)</code>	检测信号是否在信号集中

6.6 信号量（Semaphore）（※）

信号量定义：非负整数，用于控制多进程对共享资源的访问（同步机制）

三个基本操作

- **初始化：**设置初始值（表示可用资源数量）
- **P操作（wait/down）：**值>0则减1并立即返回（获取资源）；值=0则阻塞等待
- **V操作（signal/up）：**值加1（释放资源），通知其他等待进程

类比图书馆借书：

- 初始：信号量=1（书有一本）
- 学生A借书：P操作，信号量-1=0（书被借出）
- 学生B想借：信号量=0，系统提示等待（阻塞）
- 学生A还书：V操作，信号量+1=1（书归还，B可借）

信号量函数

函数	功能
<code>semget(key, nsems, flag)</code>	创建/获取信号量集
<code>semop(semid, semoparray, nops)</code>	执行P/V操作

函数	功能
<code>semctl(semid, semnum, cmd, ...)</code>	控制 (SETVAL初始化; GETVAL获取; IPC_RMID删除)

6.7 共享内存

特点：最快的IPC方式（进程直接访问同一块内存），通常需配合信号量保证数据一致性

基本步骤：创建（`shmget`）→ 连接（`shmat`）→ 访问 → 断开（`shmdt`）→ 删除（`shmctl`）

IPCS 命令：查看IPC设施状态

- `ipcs -s`：信号量；`ipcs -m`：共享内存；`ipcs -q`：消息队列

第 7 章 Linux 网络编程

7.1 网络程序基础

网络程序特点：由客户端和服务端两部分组成

- 先启动服务器端，在某个端口上监听
- 客户端发送请求后建立连接

Linux 网络编程核心：通过 **Socket（套接字）** 进行网络编程，Socket返回文件描述符，像操作文件一样收发数据（设备无关性）

OSI 七层模型（※）

层次	名称	功能
7	应用层	为网络应用提供接口（HTTP、FTP）
6	表示层	数据加密、压缩、格式转换
5	会话层	建立、管理、终止会话
4	传输层	端到端可靠传输（TCP/UDP）
3	网络层	路由选择、分组转发（IP）
2	数据链路层	帧同步、流量控制、错误控制
1	物理层	比特流传输，处理电气/机械接口

7.2 网络地址与配置（※）

IP 地址：IPv4为32位点分十进制（如 192.168.1.1），为网络中每台主机分配逻辑地址

特殊地址

- **私有地址：**内部网络用，不能在公网路由（如 192.168.x.x）
- **回环地址：**127.0.0.1，用于本机测试，数据包不离开本机
- **广播地址：**主机部分全为1（如 192.168.1.255），向网络所有设备发送

子网掩码 (※)： 网络号全1、主机号全0的32位地址，与IP按位AND得到网络地址

- 例：IP 192.168.1.100 + 掩码 255.255.255.0 → 网段 192.168.1.0/24
- /24 表示子网掩码24位

网关 (Gateway)： 不同网络间的桥梁，功能包括：

- **路由：** 决定数据包最佳路径
- **协议转换：** 翻译不同网络的"语言"
- **安全：** 过滤检查数据包，防止非法访问
- **NAT地址转换：** 使私有IP设备能访问互联网

常用网络命令

命令	功能
ping 目标	测试网络连通性，测量延迟和丢包率
netstat	查看网络连接、路由表 (-a 所有; -n 数字显示; -t TCP; -u UDP)
ifconfig	显示和配置网络接口 (安装: apt-get install net-tools)
wget URL	从网络下载文件

7.3 网络协议 (※)

协议三要素： 语法（数据格式）、语义（控制信息含义）、时序（执行顺序）

TCP/IP 四层模型： 应用层 → 传输层 → 网络层 → 链路层

核心协议对比 (※)

协议	全称	层次	特点
TCP	传输控制协议	传输层	面向连接、可靠、三次握手、字节流
UDP	用户数据报协议	传输层	无连接、不可靠、速度快、适合实时
IP	因特网互联协议	网络层	用IP地址标识每台设备
HTTP	超文本传输协议	应用层	基于TCP，用于Web资源传输，明文
HTTPS	HTTP安全版	应用层	HTTP + 传输加密 + 身份认证
ICMP	网络控制消息协议	网络层	报告错误（如主机不可达），ping使用ICMP
FTP	文件传输协议	应用层	文件传输

TCP 特点 (※)： 可靠连接，连接建立需**三次握手**

UDP 适用场景： 音频/视频流传输、实时游戏、网络会议、NTP时间同步（对实时性要求高，允许少量丢包）

IP 数据包头部字段： 版本(4b)、头部长度(4b)、服务类型(8b)、总长度(16b)、标识(16b)、标志(3b)、分段偏移量(13b)、生存时间TTL(8b)、协议(8b)、头部校验和(16b)

ICMP 消息格式： 类型(Type)字段 + 代码(Code)字段 + 校验和(Checksum)字段

7.4 Socket 网络编程函数 (※)

地址结构体

```
1 struct sockaddr_in {
2     sa_family_t  sin_family; // 地址族 (AF_INET=IPv4)
3     uint16_t     sin_port;   // 16位端口号 (网络字节序)
4     struct in_addr sin_addr;  // 32位IP地址
5     char         sin_zero[8]; // 填充 (不使用)
6 };
```

地址族: AF_INET (IPv4) ; AF_INET6 (IPv6) ; AF_UNIX (本地IPC)

核心 Socket 函数 (※)

函数	功能	返回值
socket(domain, type, protocol)	创建套接字文件描述符	fd (成功) , -1 (失败)
bind(sockfd, addr, addrlen)	绑定地址和端口	0 (成功) , -1 (失败)
listen(sockfd, backlog)	监听连接 (服务器用)	0 (成功) , -1 (失败)
accept(sockfd, addr, addrlen)	接受新连接 (服务器用)	新fd (成功) , -1 (失败)
connect(sockfd, addr, addrlen)	主动连接 (客户端用)	0 (成功) , -1 (失败)
send(sockfd, buf, len, flags)	发送数据	发送字节数, -1失败
recv(sockfd, buf, len, flags)	接收数据	接收字节数, -1失败
write(fd, buf, count)	向fd写数据	写入字节数, -1失败
read(fd, buf, count)	从fd读数据	读取字节数, 0=文件尾, -1失败
close(fd)	关闭套接字 (同关闭文件)	—
shutdown(s, how)	关闭套接字 (how=0终止读, 1终止写, 2全关)	0成功, -1失败
getsockopt / setsockopt	获取/设置套接字选项	0成功, -1失败

服务器端编程流程 (※)

```
1 socket() → bind() → listen() → while(1) { accept() → read/recv() → write/send() → close(client_fd) }
```

客户端编程流程 (※)

```
1 | socket() → connect() → write/send() → read/recv() → close()
```

并发TCP服务器伪代码

```
1 | socket(); bind(); listen();
2 | while(1) {
3 |     accept();
4 |     if (fork() == 0) { // 子进程处理客户端
5 |         while(1) { read(); process(); write(); }
6 |         close(); exit();
7 |     }
8 |     close(); // 父进程关闭客户端fd继续accept
9 | }
```

字节序转换函数 (※)

- **主机字节序**：不同硬件可能是大端或小端
- **网络字节序**：统一采用**大端字节序**
- 发送前：主机字节序 → 网络字节序 (htonl/htons)
- 接收后：网络字节序 → 主机字节序 (ntohl/ntohs)

函数	说明
htonl(hostlong)	主机序long → 网络序 (host to network long)
htons(hostshort)	主机序short → 网络序
ntohl(netlong)	网络序 → 主机序long
ntohs(netshort)	网络序 → 主机序short

域名解析 (DNS)：将域名（如www.baidu.com）转换为IP地址的过程

第 8 章 ARM 汇编

8.1 ARM 架构概述 (※)

ARM (Advanced RISC Machines)：知名微处理器架构设计公司，RISC精简指令集架构

ARM 处理器技术特征 (※)

- 体积小、**低功耗**、低成本、高性能
- 支持ARM(32位) / Thumb(16位) **双指令集**，能很好兼容8位/16位器件
- 大量使用寄存器，大多数数据操作在**寄存器**中完成
- 寻址方式灵活简单，指令长度**固定**
- 采用：RISC技术、**流水线技术**、超标量执行、高速缓存(Cache)

ARM 数据类型 (※)

- 字节(Byte): 8位
- 半字(Half Word): 16位 (2B)
- 字(Word): 32位 (4B)
- 有符号数最高位为符号位
- 仅STR/STM/LDR/LDM指令支持较短的字节和半字数据

ARM vs x86: ARM在移动设备和嵌入式领域更具优势, 功耗/性能平衡更好

ARM 家族: ARMv7、ARMv8、ARMv9等不同版本, 各有特定优化

8.2 汇编语言基础 (※)

汇编语言 (Assembly Language)

- **面向机器**的程序设计语言
- 使用**助记符**代替机器码二进制序列
- 需要**汇编器**将汇编翻译成机器语言
- 优势: 对硬件直接控制, 执行效率高
- 缺点: 复杂繁琐, 依赖具体机型, **不可移植**
- 嵌入式中用于: 初始化程序、驱动程序、执行速度要求高的程序

ARM 汇编代码三要素

- **指令:** 告诉处理器执行特定操作
- **注释:** 以分号 ; 开头, 到行末结束
- **标签 (标号):** 为代码位置命名, 以字母开头

ARM 开发工具: Keil MDK、ADS (ARM Developer Suite) 、Eclipse

源文件后缀: 汇编源文件 *.S; 引入文件 *.INC; C程序 *.C; 头文件 *.H

8.3 ARM 程序结构 (※)

ARM 汇编语句格式 (※)

```
1 | {<标号>} <指令或伪指令> { ; 注释 }
```

- 指令助记符全大写或全小写, **不允许大小写混用**
- **标签 (标号) 必须顶格书写**
- AREA、END等伪操作**不能顶格写**
- 注释以 ; 开头
- 语句太长可用 \ 连接下一行

错误示例分析 (※)

```
1  START      ; 错误：标签START没有顶格写（应从第一列开始）
2  ABC:       ; 错误：标签后不能带冒号
3      MOV R2, #3 ; 正确
4  LOOP
5      Mov R2, #3 ; 错误：大小写混合
6      B loop   ; 错误：B指令区分大小写，只有LOOP标签，没有loop标签
```

段 (Section)： ARM汇编源程序由若干段组成，每个段以 AREA 开始

- 段属性：CODE（代码段） / DATA（数据段）
- 读写属性：READONLY（只读） / READWRITE（可读写）

典型程序框架 (※)

```
1      AREA EXAMPLE1, CODE, READONLY ; 定义只读代码段
2      ENTRY                          ; 指定程序入口（第一条执行指令）
3  START
4      MOV R0, #10 ; R0 = 10
5      MOV R1, #3  ; R1 = 3
6      ADD R0, R0, R1; R0 = R0 + R1 = 13
7      END                          ; 程序结束
```

8.4 伪操作 (※)

伪操作： 汇编阶段起作用，**不生成机器码**，指示汇编器如何处理程序

伪操作	功能
AREA 段名, 属性	定义代码/数据段
ENTRY	指定程序入口（第一条执行指令的位置）
END	通知汇编器源程序结束

伪指令： 在汇编时被替换成真实ARM/Thumb指令，本身不生成机器码（与伪操作的区别：伪指令可能生成多条机器指令）

8.5 ARM 寄存器 (※)

寄存器概况： ARM共有37个32位寄存器（分为通用寄存器和特殊功能寄存器），最多同时存在18个

重要寄存器 (※)

寄存器	别名	功能
R0~R7	低8位通用寄存器	通用数据存储
R8~R12	—	通用寄存器
R13	SP（栈指针）	指向当前栈顶
R14	LR（链接寄存器）	调用子程序时保存返回地址

寄存器	别名	功能
R15	PC (程序计数器)	存放下一条将要执行的指令地址
CPSR	—	当前程序状态寄存器，记录CPU当前状态（模式、条件标志等）
SPSR	—	程序状态保存寄存器，保存CPSR内容（进入异常时）

CPSR 关键位 (※)

位	含义
bit[31] N	结果为负数
bit[30] Z	结果为零
bit[29] C	进位/借位
bit[28] V	溢出
bit[7] I	I=1禁止IRQ中断
bit[6] F	F=1禁止FIQ中断
bit[5] T	T=0：ARM状态；T=1：Thumb状态
bit[4:0]	Mode位（处理器模式）

影子式设计 (Shadow Register)： 某些模式下有自己专用的寄存器（如每个模式有独立的R13/SP），不同模式共享其他寄存器

子程序调用 (※)

```
1      BL  SUBPROG      ; 调用子程序（LR = 下一条指令地址）
2      ; ... 调用后继续执行 ...
3
4  SUBPROG
5      ; ... 子程序内容 ...
6      MOV PC, LR      ; 返回（PC = LR中保存的返回地址）
```

LR与异常： 发生异常时，LR保存异常发生前PC的值减4（以便异常处理后正确返回）

8.6 ARM 工作模式 (※)

7种工作模式

模式	类型	触发条件
用户模式(User)	普通	大部分任务执行（非特权）
快速中断模式(FIQ)	特权	高优先级 (fast) 中断
普通中断模式(IRQ)	特权	低优先级 (normal) 中断

模式	类型	触发条件
超级用户模式(SVC)	特权	复位或软中断指令执行
中止模式(Abort)	特权	存取异常（内存访问错误）
未定义模式(Undef)	特权	执行未定义指令
系统模式(System)	特权	与User模式共享寄存器集

- 同一时刻CPU只处于一种模式
- 可通过修改CPSR的Mode位实现模式切换

THUMB 指令集

- 16位指令集扩展，代码尺寸更小（更紧凑）
- 与ARM指令集共享相同的寄存器集和处理器状态
- 可通过模式切换（T位）在ARM/Thumb间切换

8.7 ARM 寻址方式（※）

寻址方式	示例	说明
立即数寻址	ADD R0, R1, #5	操作数为立即数（以 # 开头；#0x 十六进制，#0b 二进制）
寄存器寻址	MOV R0, R1	操作数全为寄存器
寄存器移位寻址	ADD R0, R1, R2, ROR #5	第二个寄存器操作数先移位再使用
寄存器间接寻址	LDR R0, [R1]	方括号内寄存器的值为操作数地址
基址变址寻址	LDR R0, [R1, #5]	基址寄存器+偏移量
相对寻址	BEQ process1	以PC为基址，标记作为偏移量
多寄存器寻址	LDMIA R0, {R1-R5}	一条指令完成多个寄存器传送（最多16个）
堆栈寻址	STMFD R13!, {R0-R4}	数据栈与寄存器组之间批量传输

8.8 数据处理指令（※）

指令格式：<opcode>{<cond>}{S} <Rd>, <Rn>, <op2>

数据传送指令

指令	格式	功能	示例
MOV	MOV{cond}{S} Rd, op2	传送 (op2→Rd)	MOV R1, #0x10 / MOV PC, LR（子程序返回）

指令	格式	功能	示例
MVN	MVN{cond}{S} Rd, op2	按位取反后传送	MVN R1, #0xFF → R1=0xFFFFF00

算术运算指令

指令	格式	功能	示例
ADD	ADD{cond}{S} Rd, Rn, op2	Rd = Rn + op2	ADD R1, R1, #1
SUB	SUB{cond}{S} Rd, Rn, op2	Rd = Rn - op2	SUBS R0, R0, #1
RSB	RSB{cond}{S} Rd, Rn, op2	Rd = op2 - Rn (逆向减法)	RSB R0, R1, #0 → R0=-R1
MUL	MUL{cond}{S} Rd, Rm, Rs	Rd = Rm × Rs (低32位)	MUL R1, R2, R3
MLA	MLA{cond}{S} Rd, Rm, Rs, Rn	Rd = Rm × Rs + Rn	MLA R3, R1, R2, R0

逻辑运算指令

指令	格式	功能	示例
AND	AND{cond}{S} Rd, Rn, op2	按位与	AND R2, R1, R3
ORR	ORR{cond}{S} Rd, Rn, op2	按位或	ORR R0, R0, #0x0F (低4位置1)
EOR	EOR{cond}{S} Rd, Rn, op2	按位异或	EOR R1, R1, #0x0F (低4位取反)
BIC	BIC{cond}{S} Rd, Rn, op2	位清除 (Rd = Rn AND NOT op2)	BIC R1, R1, #0x0F (低4位清零)

比较/测试指令（不保存结果，只更新CPSR标志位）

指令	格式	功能	说明
CMP	CMP{cond} Rn, op2	Rn - op2, 更新CPSR	与SUBS区别：不保存结果
CMN	CMN{cond} Rn, op2	Rn + op2, 更新CPSR	用于负数比较（与ADDS区别：不保存结果）
TST	TST{cond} Rn, op2	Rn AND op2, 更新CPSR	位测试（与ANDS区别：不保存结果）
TEQ	TEQ{cond} Rn, op2	Rn EOR op2, 更新CPSR	相等测试（不影响V位和C位）

条件码 (※)

条件码	助记符	标志	含义
0000	EQ	Z=1	相等
0001	NE	Z=0	不等
1010	GE	N=V	有符号大于等于
1011	LT	N≠V	有符号小于
1100	GT	Z=0,N=V	有符号大于
1101	LE	Z=1或N≠V	有符号小于等于

8.9 分支指令

指令	格式	功能
B	B{cond} label	无条件/条件跳转 (±32MB范围)
BL	BL{cond} label	带链接跳转 (保存返回地址到LR) , 用于子程序调用
BX	BX{cond} Rm	带状态切换跳转 (Rm[0]=1切换到Thumb, =0保持ARM)

存储器访问指令

指令	功能	示例
LDR	从内存加载到寄存器	LDR R0, [R1]; LDR R0, [R1, #5]
STR	从寄存器存储到内存	STR R0, [R1]
LDMIA	多寄存器加载 (地址递增)	LDMIA R0, {R1-R5}
STMFD	多寄存器压栈 (满递减栈)	STMFD R13!, {R0-R4}

第 9 章 嵌入式环境搭建

9.1 嵌入式系统与 Linux 内核

什么是嵌入式 (概念回顾)：以应用为中心的专用计算机系统

Linux 内核：Linux操作系统的核心部分，负责管理硬件和软件资源

Linux 内核主要功能

功能	说明
进程管理	创建、调度、终止进程，管理IPC
内存管理	管理物理内存和虚拟内存

功能	说明
设备驱动	与磁盘、网卡、键盘等硬件通信
文件系统	支持ext4、XFS等，提供文件访问接口
网络协议栈	实现TCP/IP、UDP等协议

历史：1991年由 **Linus Torvalds** 开发，开源（GNU GPL），现为最流行的OS内核，**Linus至今仍是主要开发者**

9.2 内核版本类型（※）

类型	说明
Prepatch（RC，预发行）	面向开发者测试，由Linus Torvalds维护
Mainline（主线）	由Linus维护，每2~3个月发布新版本
Stable（稳定分支）	主线发布后的稳定版，有错误修复
Longterm（长期维护）	长期提供重要错误修复支持

版本编号： 主版本号.次版本号.修订版本号

- 例：5.10.214：主版本5，次版本10（功能更新），修订版本214（错误修复）

9.3 内核编译工具与流程（※）

主要编译工具

工具	作用
GCC （GNU Compiler Collection）	主要编译器，将C/汇编转为可执行内核映像； <code>gcc a.c -o a</code>
Make	自动化构建工具，根据Makefile执行编译命令
<code>make menuconfig</code>	图形化内核配置界面（选择支持的功能）
<code>make -jN</code>	N线程并行编译（N通常为CPU核心数）

查询CPU核心数： `nproc` 或 `lscpu` 或 `cat /proc/cpuinfo | grep "processor"`

编译简易 x86_64 Linux 内核流程（※）

```
1 # 安装依赖
2 sudo apt-get install -y build-essential make bison flex libncurses5
  libncurses5-dev
3 sudo apt-get install -y libelf-dev libssl-dev
4
5 # 准备源码
6 tar -xvf linux-4.19.311.tar.xz -C ~/linuxx86
7 cd ~/linuxx86/linux-4.19.311
8
```

```
9  # 配置
10 export ARCH=x86                # 设置目标架构
11 make x86_64_defconfig          # 使用默认配置
12 make menuconfig                # 图形化配置（可选）
13
14 # 编译（4核并行）
15 make -j4
16 # 也可写成：make -j`nproc`（自动使用最大核心数）
```

查看内核版本： `uname -r` 或 `uname -a`

配置技巧：将当前系统的内核配置作为起点

```
1 | sudo cp /boot/config-$(uname -r) ~/linux-xxx/.config
```

内核目录结构（关键）

目录	内容
arch/<arch>	各处理器体系结构特有代码（x86, arm, arm64, mips等）
drivers/	内建驱动代码
fs/	文件系统
ipc/	进程间通信
kernel/	通用内核代码
mm/	内存管理
net/	网络协议代码
include/	内核头文件
init/	初始化程序（含main.c）
Makefile	顶层编译说明文件

9.4 根文件系统（rootfs）（※）

rootfs 定义：Linux操作系统核心组件，包含系统启动和运行所需的基本文件和目录结构

rootfs 目录结构（※）

目录	内容
/bin	系统启动所需核心命令（二进制可执行文件）
/boot	引导文件（Linux内核映像、启动配置）
/dev	设备文件（与硬件设备交互）
/etc	系统配置文件（网络、服务、账号信息）
/home	每个用户的主目录

目录	内容
/proc	虚拟文件系统，包含进程和系统信息
/media /mnt	挂载可移动设备和其他文件系统

rootfs 构建工具 (※)

工具	说明
BusyBox	集成众多Linux命令工具 (ls/cat/grep/find等)，被称为"Linux工具界的瑞士军刀"
Debootstrap	Debian/Ubuntu下构建精简根文件系统，目录结构遵循FHS标准

BusyBox 编译配置

```
1 # 静态编译（不需要额外动态链接库）
2 make menuconfig
3 # 选中: Busybox Settings → Build Options → [*] Build static binary
4
5 make -j4 && make install -j4
6 # 生成的根文件系统位于 _install 目录
```

完善 rootfs (关键脚本)

```
1 mkdir -p dev mnt proc sys tmp etc/init.d/
2 # etc/fstab: 定义系统启动时自动挂载的文件系统
3 # etc/init.d/rcS: 系统启动时执行的初始化脚本
4 # etc/inittab: init进程配置
5
6 # 创建必要设备文件
7 cd dev
8 sudo mknod console c 5 1
9 sudo mknod null c 1 3
10 sudo mknod tty1 c 4 1
```

9.5 QEMU 仿真环境

QEMU：通用、开源的机器仿真和虚拟化软件，支持3种模式：

模式	命令	说明
全系统仿真	qemu-system-xxx	仿真完整硬件，运行完整OS
用户模式仿真	qemu-user	在当前OS下运行其他架构的Linux程序
虚拟化	—	以接近原生性能运行KVM/Xen虚拟机

安装QEMU

```

1 | sudo apt-get install -y qemu-system-x86    # x86仿真
2 | sudo apt-get install -y qemu-system-arm    # ARM仿真
3 | sudo apt-get install -y qemu-user          # 用户模式

```

启动x86内核 (示例)

```

1 | qemu-system-x86_64 \
2 |   -kernel ~/linuxx86/linux-4.19.311/arch/x86_64/boot/bzImage \
3 |   -initrd ~/linuxx86/rootfs.img.gz \
4 |   -append "root=/dev/ram init=/linuxrc console=ttyS0" \
5 |   -serial mon:stdio -nographic
6 | # 关机: poweroff; 强退: Ctrl+A 然后 X

```

启动ARM开发板 (vexpress-a9)

```

1 | qemu-system-arm -M vexpress-a9 \
2 |   -m 512M -kernel ~/boot/zImage \
3 |   -dtb ~/boot/vexpress-v2p-ca9.dtb \
4 |   -nographic \
5 |   -append "root=/dev/mmcb1k0 console=ttyAMA0" \
6 |   -sd ~/rootfs.ext3

```

用户模式运行ARM程序

```

1 | qemu-arm -L /usr/arm-linux-gnueabihf ./b    # b是ARM架构可执行文件

```

9.6 交叉编译工具链 (※)

交叉编译: 在A机器 (x86宿主机) 上生成可在B机器 (ARM目标机) 上运行的代码

好处: 嵌入式系统资源有限 (CPU主频低、内存小), 无法本地编译, 利用宿主机强大算力

安装 ARM 交叉编译工具链

```

1 | sudo apt-get install gcc-arm-linux-gnueabihf \
2 |   cpp-arm-linux-gnueabihf \
3 |   g++-arm-linux-gnueabihf
4 | arm-linux-gnueabihf-gcc -v    # 验证安装

```

使用交叉编译器

```

1 | gcc a.c -o a                # 编译 x86 程序 (本地)
2 | arm-linux-gnueabihf-gcc a.c -o b    # 交叉编译 ARM 程序
3 | file a                          # 查看可执行文件架构
4 | ldd a                          # 查看依赖库

```

编译 ARM Linux 内核

```

1 export ARCH=arm
2 export CROSS_COMPILE=arm-linux-gnueabihf-
3 make vexpress_defconfig          # 使用 vexpress 开发板默认配置
4 make uImage LOADADDR=0x60000000 -j4 # 编译内核
5 make dtbs modules -j4           # 编译设备树和模块

```

创建 rootfs ext3 文件系统镜像

```

1 dd if=/dev/zero of=rootfs.ext3 bs=1M count=32 # 创建32M空文件
2 mkfs.ext3 rootfs.ext3                        # 格式化为ext3
3 mkdir fs && sudo mount -o loop rootfs.ext3 ./fs # 挂载
4 sudo cp -rf busybox-1.32.0/_install/* ./fs    # 复制BusyBox
5 sudo umount ./fs
6 gzip --best -c rootfs.ext3 > ./rootfs.img.gz  # 压缩

```

9.7 Bootloader (※)

Bootloader (启动引导程序)：嵌入式Linux系统中，在操作系统运行之前执行的程序

6大主要功能 (※)

1. **初始化硬件设备**：处理器寄存器设置、内存控制器配置、总线激活等
2. **加载操作系统内核**：从Flash/ROM读取内核映像并加载到内存
3. **设置内核启动参数**：传递硬件配置、内存布局等信息
4. **提供调试接口**：串口、JTAG等，方便开发调试
5. **支持固件升级**：接收新固件并写入非易失性存储器（在线升级）
6. **多重引导支持**：支持多个OS或应用程序的引导选择

Bootloader 启动流程：硬件初始化 → 加载自身 → 内存映射和配置 → 加载和设置内核 → 跳转到内核入口点

常见 Bootloader 类型

名称	说明
U-Boot	最常见，支持ARM/MIPS/PowerPC等多种架构，高度可配置
RedBoot	—
vivi	—

U-Boot 编译 (ARM)

```

1 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- vexpress_ca9x4_defconfig
2 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j4

```

U-Boot 手动启动命令 (4条)

```
1 # 设置启动参数
2 setenv bootargs "root=/dev/mmcblk0p2 rw console=ttyAMA0 init=/linuxrc"
3 # 从mmc加载内核到内存
4 fatload mmc 0:1 0x60008000 uImage
5 # 从mmc加载设备树
6 fatload mmc 0:1 0x62008000 vexpress-v2p-ca9.dtb
7 # 引导内核
8 bootm 0x60008000 - 0x62008000
```

uEnv.txt: U-Boot环境变量配置文件，位于FAT分区根目录，可实现自动开机（vexpress_a9不支持，需修改源码）

作业题

题 1 Linux 基本命令操作

题目

在 `/home/aistudio/` 目录（如没有则创建）下创建临时目录 `test`，进入 `test` 目录，创建 `.a`、`b` 文件，创建 `c`、`d` 文件夹，用命令显示以上4条内容（含隐藏文件），并显示当前所在目录。

答案

```
1 mkdir -p /home/aistudio/test
2 cd /home/aistudio/test
3 touch .a b
4 mkdir c d
5 ls -a
6 pwd
```

题目

创建 `a.txt`，用vi编辑写入两行：

```
1 Reading makes people perfect.
2 knowledge is power.
```

使用一行命令抓取并打印 `a.txt` 中带有 `power` 的行。

答案

```
1 cat a.txt | grep power
```

题 2 Shell 脚本编程

题目1: 编写 shell 脚本，在当前目录创建 `test` 文件夹，在该目录中创建 `hello.txt` 文件，并写入 `Hello world!`。

答案

```

1  #!/bin/bash
2  mkdir test
3  touch test/hello.txt
4  echo "Hello world!" > test/hello.txt

```

题目2: 编写 shell 脚本，接收两个参数分别存入变量 `a` 和 `b`，将这两个参数相加并输出结果。

答案

```

1  #!/bin/bash
2  a=$1
3  b=$2
4  echo $(( a+b ))

```

题 3 文件 IO 编程 (C语言)

题目1: 使用C语言库函数创建文件 `text.txt`，写入 `HBUT_Science_EmbeddedSystem`，读取并打印到屏幕。

答案

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  int main() {
4      char *filename = "text.txt";
5      char *text = "HBUT_Science_EmbeddedSystem";
6      /* 写入 */
7      FILE *file = fopen(filename, "w");
8      fwrite(text, 1, 27, file);
9      fclose(file);
10     /* 读取并打印 */
11     char buffer[64];
12     FILE *file2 = fopen(filename, "r");
13     fgets(buffer, sizeof(buffer), file2);
14     printf("%s\n", buffer);
15     fclose(file2);
16     return 0;
17 }

```

题目2: 使用C语言库函数打印"日历时间"（秒数）和当前的"年-月-日"。

答案

```

1  #include <time.h>
2  #include <stdio.h>
3  void main() {
4      time_t timep;
5      struct tm *p;
6      time(&timep);
7      printf("%d\n", timep); /* 日历时间（从1970-01-01 00:00:00到现在的秒数） */
8      p = gmtime(&timep);
9      printf("%d-%d-%d\n", (1900+p->tm_year), (1+p->tm_mon), p->tm_mday);
10 }

```

题 4 ARM 汇编编程

题目1: 把 0x10 和 0x40 分别存储在 R0 和 R1 中，将 R0 与 0xF 进行逻辑或结果保存至 R2，将 R1 的值与 R2 相加并将结果存储在 R2。

答案

```
1      AREA RESET, CODE, READONLY
2      ENTRY
3  START
4      MOV R0, #0x10      ; R0 = 0x10 = 16
5      MOV R1, #0x40      ; R1 = 0x40 = 64
6      ORR R2, R0, #0xF    ; R2 = R0 | 0xF = 0x10 | 0x0F = 0x1F
7      ADD R2, R1, R2      ; R2 = R1 + R2 = 0x40 + 0x1F = 0x5F
8      END
```

题目2: 由子程序 doadd 完成加法运算，操作数10和3分别放在 R0 和 R1 寄存器中，结果存在 R2。

答案

```
1      AREA RESET, CODE, READONLY
2      ENTRY
3  START
4      MOV R0, #10        ; 设置输入参数 R0 = 10
5      MOV R1, #3         ; 设置输入参数 R1 = 3
6      BL doadd           ; 调用子程序 doadd (LR = 下一条指令地址)
7      B .               ; 无限循环，程序在此停留
8
9  doadd
10     ADD R2, R0, R1      ; R2 = R0 + R1 = 13 (子程序功能)
11     MOV PC, LR         ; 从子程序返回 (PC = LR中保存的返回地址)
12
13     END
```